```
Content-Type: application/json
X-Content-Type-Options: nosniff
X-XSS-Protection: 1; mode=block
Cache-Control: private, max-age=0
Server:
Transfer-Encoding: chunked

{"token":"node0hwk7s0nq6wvppqh0wbs0cha91"}    #C
```

#A Use the -u option to send HTTP Basic credentials
#B Spark returns a Set-Cookie header for the new session token
#C The TokenController also returns the token in the response body

4.3.1 Avoiding session fixation attacks

The code you've just written suffers from a subtle but widespread security flaw that affects all forms of token-based authentication, known as a *session fixation attack*. After the user authenticates, the `CookieTokenStore` then asks for a new session by calling `request.session(true)`. If the request did not have an existing session cookie, then this will create a new session. But if the request already contains an existing session cookie, then Spark will return that existing session and not create a new one. This can create a security vulnerability if an attacker is able to inject their own session cookie into another user's web browser. Once the victim logs in, the API will change the username attribute in the session from the attacker's username to the victim's username. The attacker's session token now allows them to access the victim's account, as shown in figure 4.8.